

CPSI 28003 - 01 and 9S1: Algorithms

Homework 3

Due at the beginning of the class, Tuesday, November 25, 2025

Submitted to BlackboardBy
Dr. Chia-Chu ChiangDepartment of Computer Science
University of Arkansas at Little Rock
2801 S. University Avenue
Little Rock, Arkansas 7204-1099, USA

79

Total Points: 100

Multiple Choice & Short Answers (6 Points)

- (2 Points) In the process of infix to postfix conversion, what happens when an operand is read in from the infix expression,
 - pushed onto the operator stack
 - placed on to the output
 - ☒ an operator on the top of the stack placed to the output
 - do nothing
- (2 Points) In the conversion of infix to postfix, what happens when a opening parenthesis '(' is read in from the infix expression,
 - read the next item from the infix expression
 - placed in the output
 - ☒ pushed onto the operator stack
 - do nothing
- (2 Points) Which of the following statement is incorrect regarding infix to postfix conversion?
 - operand is always placed in the output
 - ☒ operator is always pushed onto the operator stack
 - the opening and closing parenthesis is are always pushed onto the operator stack
 - the operators are associated from left to right

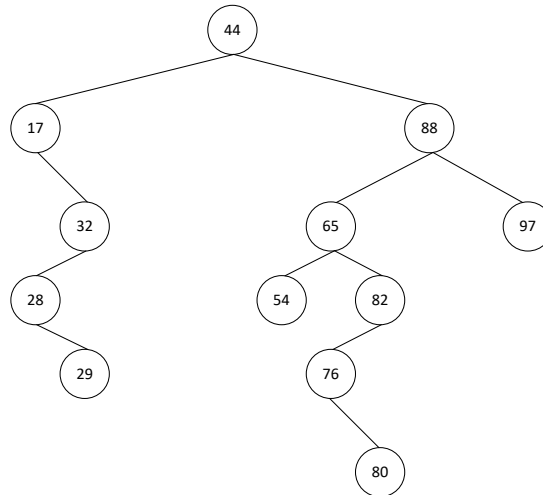
Short Answers (4 Points)

- (4 Points) Convert $a + b * (c / d - e) * (f + g * h) - i$ to postfix expression?
[ANSWER]

a b c d / 5 - * 6 7 8 * + * + 9 -

- 18

5. (8 Points) For the tree shown below,



- a. (2 Points) Which node is the root? **44**
- b. (2 Points) Which nodes are leaves? **29, 80, 54, 97**
- c. (2 Points) What is the level of the tree (Level begins with 0)? **44**
- d. (2 Points) What is the height of the tree (Height begins with 1)? **80**
6. (5 Points) Let T be a **full binary tree** with 63 internal nodes. Height begins with 1. What is the maximum and minimum heights it can have?
[ANSWER]

$$2 \cdot 63 + 1 \leq 63 \leq 2^{(63+1)} - 1$$

$$126 \leq 63 \leq 1.84 \cdot 10^{19}$$

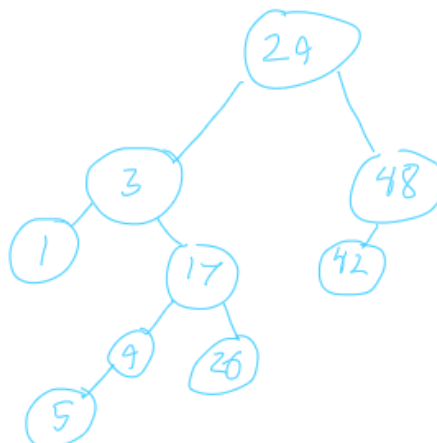
$$\text{max} = 1.84 \cdot 10^{19}$$

$$\text{min} = 126$$

7. (25 Points) Binary Search Trees.

- a. (10 Points) Insert the values 29, 3, 17, 9, 48, 26, 1, 42, and 5 into an initially empty binary search tree. The elements must be inserted in the given order.

[ANSWER]



- b. (5 Points) List the nodes of the above binary search tree in the pre-order traversal.

[ANSWER] 29 3 1 17 9 5 26 48 42

- c. (5 Points) List the nodes of the above binary search tree in the in-order traversal.

[ANSWER] 1 3 5 9 26 29 42 48



- d. (5 Points) List the nodes of the above binary search tree in the post-order traversal.

[ANSWER] 1 5 9 26 17 3 42 48 29

8. (5 Points) What does the following function f do? The parameter r is the root of a **Binary Search Tree**? **DO NOT JUST EXPLAIN THE CODE.**

```
int f(Node* r)
{
    if (r.left == NULL && r.right == NULL)
        return 0;
    else
        return 1+f(r.left)+f(r.right);
}
```

[ANSWER]

DO NOT JUST EXPLAIN THE SYNTAX OF THE CODE.

It returns the sum of the binary search tree by adding the left and right sides together.

9. (5 Points) What does the following code f do on a **Binary Search Tree**? DO NOT JUST EXPLAIN THE CODE.

```
Node* BinarySearchTree::f() const
{
    Node *tNode;
    tNode = root;
    if (tNode != NULL) {
        while (tNode->right != NULL)
            tNode = tNode->right;
    };
    return (tNode);
}
```

[ANSWER]

DO NOT JUST EXPLAIN THE SYNTAX OF THE CODE.

It traverses the binary search tree until it goes to the rightmost node and returns the rightmost node

10. (5 Points) What does the following code f do? The parameter r is the root of a **Binary Search Tree**? **DO NOT JUST EXPLAIN THE CODE.**

```
int f(node* r)
{
    if (r == NULL)
        return 0;
    else if (r->left == NULL && r->right==NULL)
        return 0;
    else
        return max(f(r->left)+1, f(r->right)+1);
}
```

[ANSWER]

DO NOT JUST EXPLAIN THE SYNTAX OF THE CODE.

The following code checks the binary search tree for the max value using preorder traversal (root, left, then right)

11. (5 Points) What does the following code f do? The parameter r is the root of a **Binary Search Tree**? **DO NOT JUST EXPLAIN THE CODE.**

```
Node* BinarySearchTree::f(Node* p) const
{
    if (p == NULL)
        return NULL;
    else{
        while (p->left != NULL)
            p = p->left;
    }
    return p;
}
```

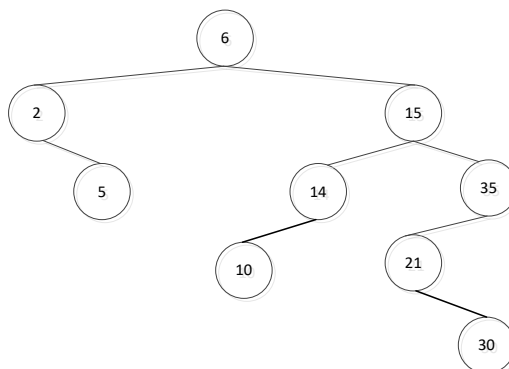
[ANSWER]

DO NOT JUST EXPLAIN THE SYNTAX OF THE CODE.

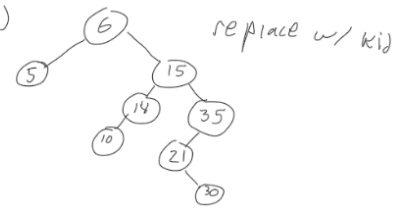
It returns the leftmost (smallest) node

12. (2 Points) What is the time complexity of the stack operation pop()? O(1)

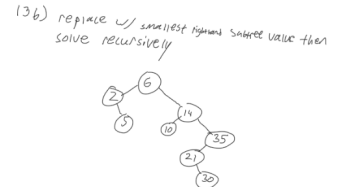
13. (10 Points) Given the following **Binary Search Tree**,



- 13a. (5 Points) Draw the **Binary Search tree** after deleting the value 2. Use one of the methods discussed in the class for deleting the node
[ANSWER]



- 13b. (5 Points) Draw the **Binary Search tree** after deleting the value 15. Use one of the methods discussed in the class for deleting the node.
[ANSWER]



14. (6 Points) What is the order of growth of the worst-case running operation below? Write down the best answer in the space provided, using one of the following possibilities.

$O(1)$ $O(\log N)$ $O(N)$ $O(N \log N)$ $O(N^2)$

- (a) $O(N)$ insert an item to a hash table
(b) $O(\log N)$ insert an item to a binary search tree
(c) $O(1)$ push an item onto a stack

9

15. (10 Points) Draw the final result after inserting keys 24, 25, 151, 38, 112, 9, 73, 24, 19, 11, 44, 80, 81 into a hash table with collisions resolved by (a) linear probing, (b) chaining. Let the table have 13 slots with addresses starting at 0, and let the hash function be $h(k) = k \bmod 13$.

- (a) (5 Points) Linear Probing

0	1	2	3	4	5	6	7	8	9	10	11	12

[ANSWER]

0	1	2	3	4	5	6	7	8	9	10	11	12
38	73	24	11	80	44	6	81	151	112	9	24	25

- (b) (5 Points) Separate Chaining
[ANSWER]

2 = 80
3 = 81
5 = 44
6 = 19
8 = 151 → 112 → 73
9 = 9
11 = 24 → 24 → 11
12 = 25 → 38

16. (4 Points) What is the output, as main() is executed,

```
#include <iostream>
using namespace std;

#include "Stack.h"

int main()
{
    Stack s(2);

    if (!s.full())
        s.push(1);
    if (!s.full())
        s.push(2);
    if (!s.full())
        s.push(3);

    if (!s.empty())
        cout << s.top() << endl;
    else
        cout << "Stack is empty!\n";

    if (!s.empty())
        s.pop();

    if (!s.empty())
        cout << s.top() << endl;
    else
        cout << "Stack is empty!\n";

    if (!s.empty())
        s.pop();

    if (!s.empty())
        cout << s.top() << endl;
    else
        cout << "Stack is empty!\n";

    system("PAUSE");
    return 0;
}
```

[ANSWER]

2

1

Stack is empty!

Stack is empty!